COMPTE RENDU DE TESTS

Tests des endpoints de VehiculeController

API REST Spring Boot – Projet AutoLoc (gestion de location de véhicules)

Projet	AutoLoc – module Véhicule
Réalisé par	Samar Ben Ahmed
Date	16/10/2026
Outil de test	Postman
Résultat global	Les 5 endpoints testés retournent 200 OK avec le comportement attendu

1. Architecture du module Véhicule

Le module respecte une architecture en couches. Chaque couche a une responsabilité unique, ce qui facilite la maintenance et les tests. Le code complet de chaque couche est fourni en annexe.

Couche	Classe / interface	Rôle
Repository	VehiculeRepository	Accès aux données : étend JpaRepository<Vehicule, Long> et fournit les opérations CRUD sans code supplémentaire.
Service (contrat)	IVehiculeService	Déclare les opérations métier : ajout, modification, consultation, liste et suppression.
Service (implémentation)	IVehiculeServiceImp	Implémente le contrat en s'appuyant sur le repository (annotation @Service).
Controller	VehiculeController	Expose les endpoints REST sous /vehicules (annotation @RestController) et délègue au service.

Le flux d'une requête est donc : Postman → VehiculeController → IVehiculeService (IVehiculeServiceImp) → VehiculeRepository → base MySQL.

2. Démarche de test

Les tests suivent un ordre logique qui respecte les dépendances entre les données :

- **Étape 1 – Prérequis :** création d'une agence, car un véhicule doit obligatoirement lui être rattaché.
- **Étape 2 – Création :** ajout d'un véhicule (POST).
- **Étape 3 – Lecture :** consultation du véhicule par son identifiant (GET).
- **Étape 4 – Modification :** mise à jour d'un champ du véhicule (PUT).
- **Étape 5 – Suppression :** suppression du véhicule (DELETE).
- **Étape 6 – Vérification finale :** lecture de la liste complète pour confirmer la suppression (GET).

3. Tests des endpoints

3.1 Prérequis : création d'une agence

Un véhicule est lié obligatoirement à une agence (clé étrangère agence_id non nulle). Une agence a donc été créée au préalable, puis son existence a été vérifiée.

Requête	POST http://localhost:9090/agences/ajouterAgence
Corps JSON	{ "nom": "Agence Tunis", "ville": "Tunis" }
Résultat	200 OK – 236 ms – 247 B

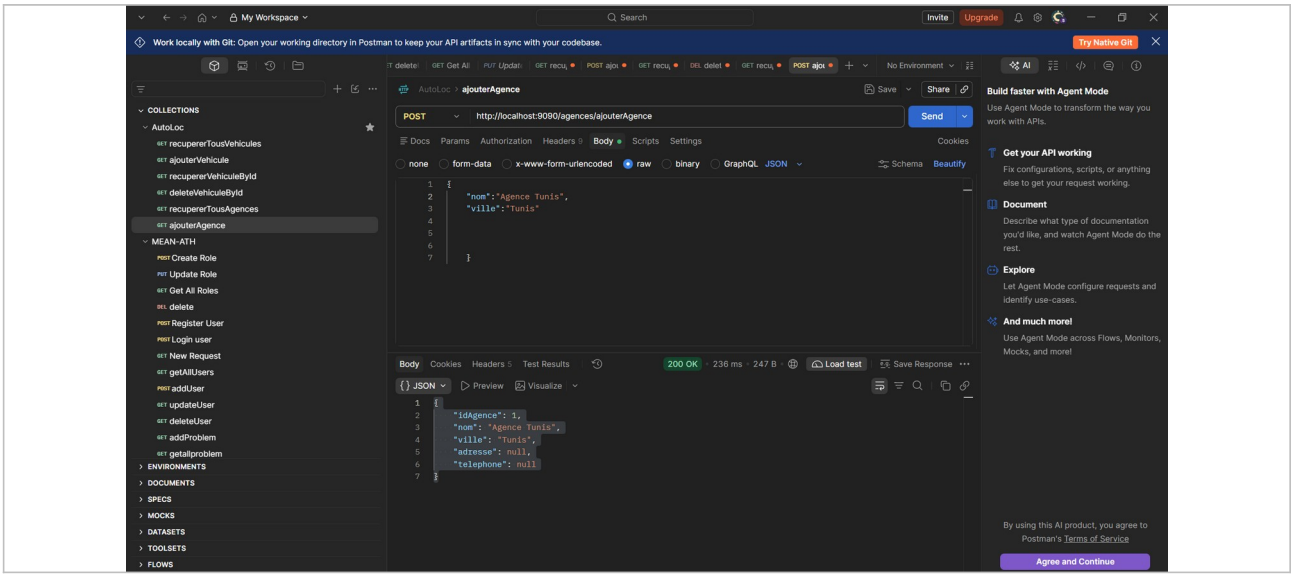


Figure 1 : POST /agences/ajouterAgence (200 OK)

Explication de la capture

- **Requête :** méthode POST sur /agences/ajouterAgence, avec un corps JSON (mode raw) contenant uniquement le nom « Agence Tunis » et la ville « Tunis ».
- **Réponse :** statut 200 OK en 236 ms. Le serveur renvoie l'agence enregistrée avec un identifiant généré automatiquement, idAgence = 1.
- **Champs à null :** adresse et telephone valent null car ils n'ont pas été fournis dans la requête ; ils ne sont pas obligatoires.
- **Conclusion :** l'agence d'id 1 existe désormais et pourra servir de clé étrangère pour le véhicule.

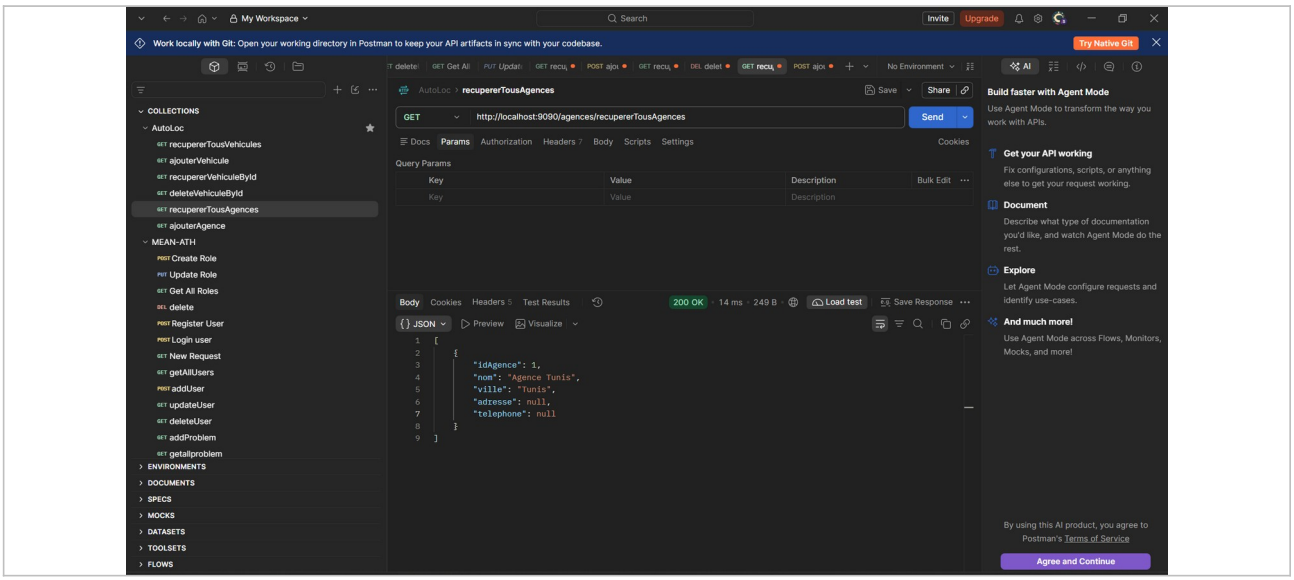


Figure 2 : GET /agences/recupererTousAgences (200 OK)

Explication de la capture

- **Requête :** méthode GET sur /agences/recupererTousAgences, sans paramètre ni corps (l'onglet Params est vide).
- **Réponse :** statut 200 OK en 14 ms. Le corps est un tableau JSON contenant une seule agence : idAgence = 1, nom « Agence Tunis », ville « Tunis ».
- **Conclusion :** l'agence créée à l'étape précédente est bien persistée en base ; le prérequis est satisfait et les tests sur les véhicules peuvent commencer.

3.2 Ajouter un véhicule (POST)

Requête	POST http://localhost:9090/vehicules/ajouterVehicule
Corps JSON	immatriculation « 123 tunis 456 », marque « Peugeot », modèle « 208 », catégorie « CITADINE », tarifJournalier 55.50, agence { idAgence : 1 }
Résultat	200 OK – 113 ms – 410 B

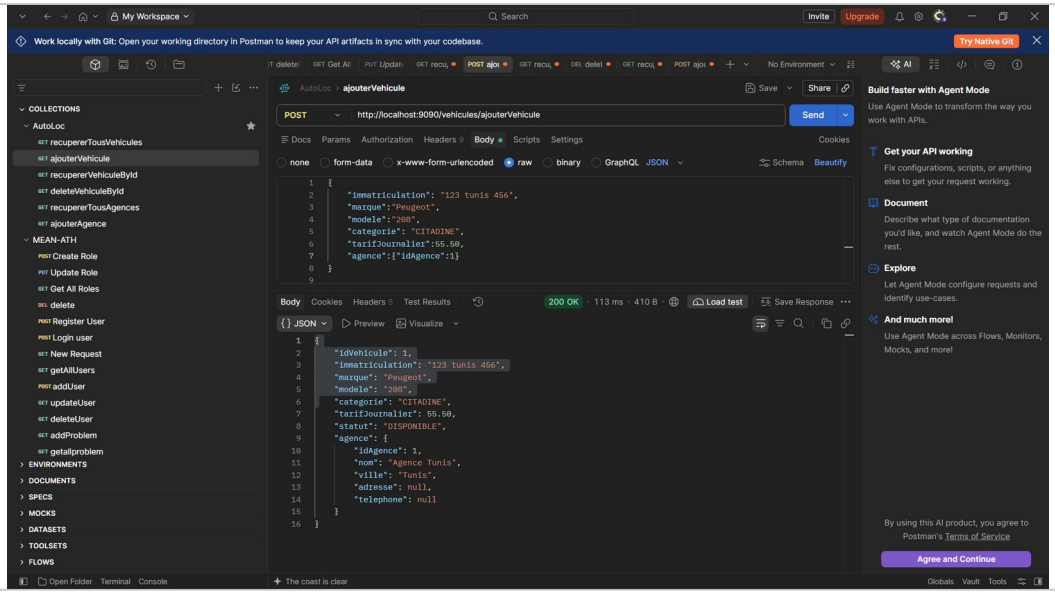


Figure 3 : POST /vehicules/ajouterVehicule (200 OK)

Explication de la capture

- **Requête :** le corps JSON décrit le véhicule (immatriculation, marque, modèle, catégorie, tarif journalier) et désigne son agence par son seul identifiant : "agence": { "idAgence": 1 }. Le contrôleur reçoit cet objet grâce à @RequestBody.
- **Identifiant généré :** la réponse contient idVehicule = 1, attribué automatiquement par la base de données.
- **Statut par défaut :** le champ statut n'a pas été envoyé, mais la réponse indique « DISPONIBLE » : c'est la valeur par défaut de l'entité pour un véhicule nouvellement créé.
- **Relation avec l'agence :** la réponse inclut l'agence complète (nom, ville, etc.) alors que seul son id a été transmis, ce qui confirme que la liaison avec l'entité Agence fonctionne.
- **Conclusion :** la création est validée : le véhicule est enregistré et retourné avec toutes ses informations.

3.3 Récupérer un véhicule par id (GET)

Requête	GET http://localhost:9090/vehicules/recupererVehicule/1
Paramètre	id = 1, passé dans le chemin de l'URL (@PathVariable)
Résultat	200 OK – 18 ms – 410 B

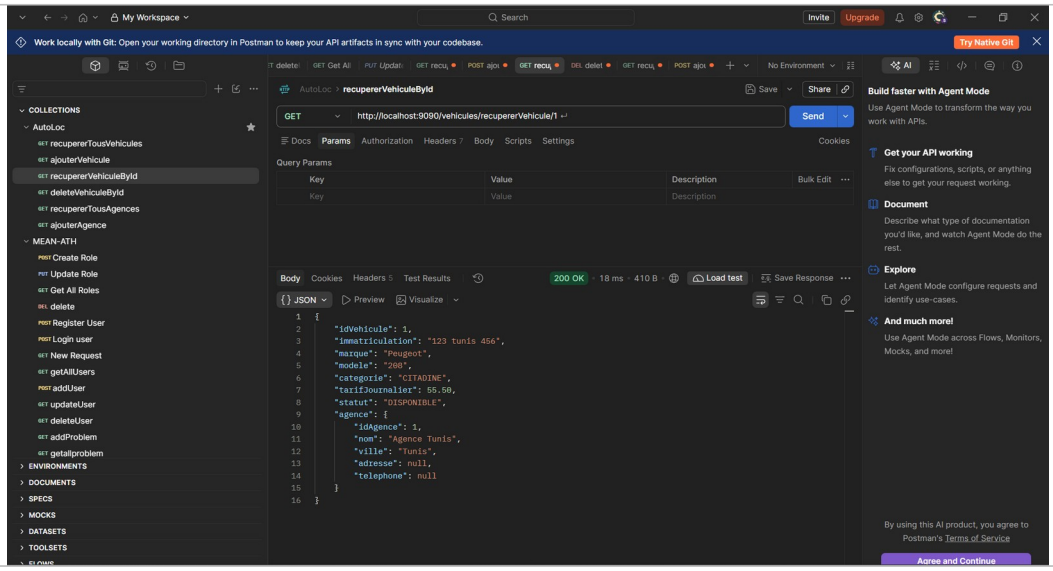


Figure 4 : GET /vehicules/recupererVehicule/1 (200 OK)

Explication de la capture

- **Requête :** l'identifiant du véhicule est directement intégré à l'URL (/recupererVehicule/1) ; c'est pourquoi l'onglet Params de Postman reste vide et qu'aucun corps n'est envoyé.
- **Réponse :** statut 200 OK en 18 ms. Le véhicule d'id 1 est retourné avec les mêmes données que celles enregistrées à l'étape précédente (« 123 tunis 456 », Peugeot 208, CITADINE, 55.50, DISPONIBLE).
- **Agence :** les informations de l'agence rattachée (id 1, « Agence Tunis ») sont incluses dans la réponse.
- **Conclusion :** la lecture par identifiant fonctionne et prouve que les données sont bien relues depuis la base.

3.4 Modifier un véhicule (PUT)

Requête	PUT http://localhost:9090/vehicules/modifierVehicule
Corps JSON	idVehicule 1, immatriculation « 145 tunis 456 » (au lieu de « 123 tunis 456 »), autres champs inchangés, agence { idAgence : 1 }
Résultat	200 OK – 551 ms – 410 B

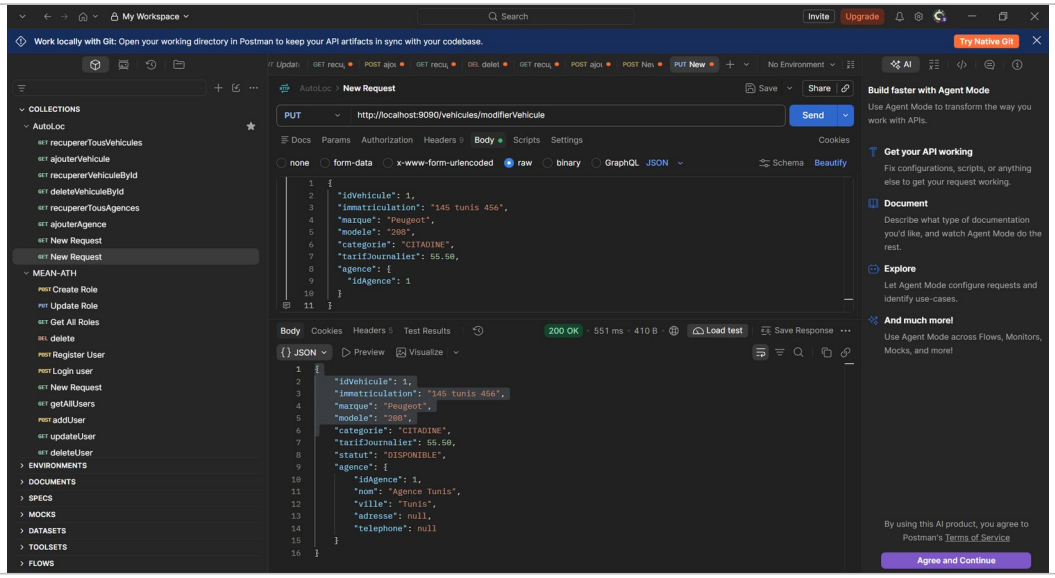


Figure 5 : PUT /vehicules/modifierVehicule (200 OK)

Explication de la capture

- **Requête :** le corps JSON reprend l'intégralité du véhicule en y ajoutant idVehicule = 1, qui désigne l'enregistrement à modifier. Seule l'immatriculation change : « 123 tunis 456 » devient « 145 tunis 456 ».
- **Fonctionnement :** le service appelle save() du repository ; comme l'identifiant existe déjà, JPA effectue une mise à jour de la ligne existante au lieu d'en créer une nouvelle.
- **Réponse :** statut 200 OK. Le véhicule retourné conserve idVehicule = 1, affiche la nouvelle immatriculation « 145 tunis 456 » et garde son statut DISPONIBLE ainsi que son agence (id 1).
- **Durée :** 551 ms, plus longue que les autres requêtes ; cela s'explique par le fait qu'une modification demande une lecture puis une écriture en base.
- **Conclusion :** la modification est validée : la valeur a changé, l'identifiant et la relation avec l'agence sont conservés.

3.5 Supprimer un véhicule (DELETE)

Requête	DELETE http://localhost:9090/vehicules/deleteVehiculeById/1
Paramètre	id = 1, passé dans le chemin de l'URL (@PathVariable)
Résultat	200 OK – 13 ms – corps vide

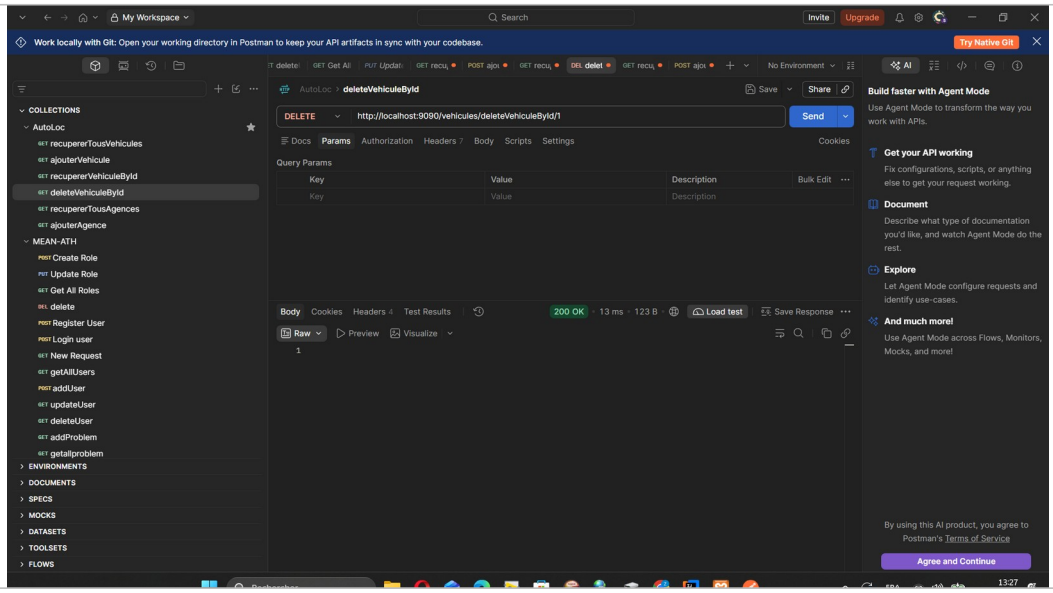


Figure 6 : DELETE /vehicules/deleteVehiculeById/1 (200 OK)

Explication de la capture

- **Requête :** méthode DELETE avec l'identifiant du véhicule dans l'URL ; aucun corps n'est nécessaire.
- **Réponse :** statut 200 OK en 13 ms avec un corps vide (la zone de réponse ne contient qu'une ligne vide). C'est le comportement attendu : la méthode supprimerVehiculeById du contrôleur est de type void et ne retourne donc aucune donnée.
- **Conclusion :** le statut 200 indique que l'opération s'est déroulée sans erreur ; l'effet réel sur la base est confirmé par le test suivant.

3.6 Récupérer tous les véhicules (GET)

Requête	GET http://localhost:9090/vehicules/recupererTousVehicules
Résultat	200 OK – 24 ms – réponse : [ ] (liste vide)

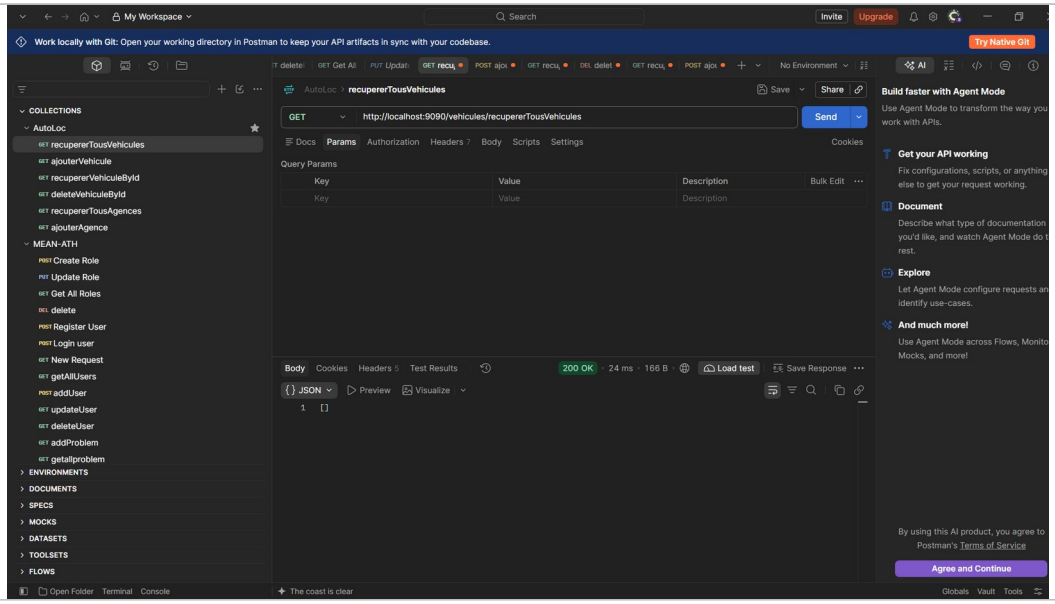


Figure 7 : GET /vehicules/recupererTousVehicules (200 OK)

Explication de la capture

- **Requête :** méthode GET sur /recupererTousVehicules, sans paramètre ni corps.
- **Réponse :** statut 200 OK en 24 ms. Le corps est un tableau JSON vide ([ ]), alors qu'un véhicule existait avant l'étape de suppression.
- **Conclusion :** la liste vide confirme que le véhicule d'id 1 a bien été supprimé de la base de données. Ce test valide à la fois l'endpoint de liste et l'effet réel du DELETE.

4. Récapitulatif des résultats

N°	Méthode	Endpoint	Objectif	Statut	Temps	Capture
1	POST	/vehicules/ajouterVehicule	Création d'un véhicule	200 OK	113 ms	Fig. 3
2	GET	/vehicules/recupererVehicule/1	Lecture par id	200 OK	18 ms	Fig. 4
3	PUT	/vehicules/modifierVehicule	Modification	200 OK	551 ms	Fig. 5
4	DELETE	/vehicules/deleteVehiculeById/1	Suppression	200 OK	13 ms	Fig. 6
5	GET	/vehicules/recupererTousVehicules	Liste complète	200 OK	24 ms	Fig. 7

Les tests préalables sur les agences (Figures 1 et 2) ont également retourné 200 OK.

Annexe : code source du module Véhicule

Les captures ci-dessous montrent le code de chaque couche dans l'IDE IntelliJ IDEA. Le panneau de gauche présente l'organisation du projet en packages (controller, entity, exception, repository, service) sous tn.esprit.spring.

A. VehiculeRepository

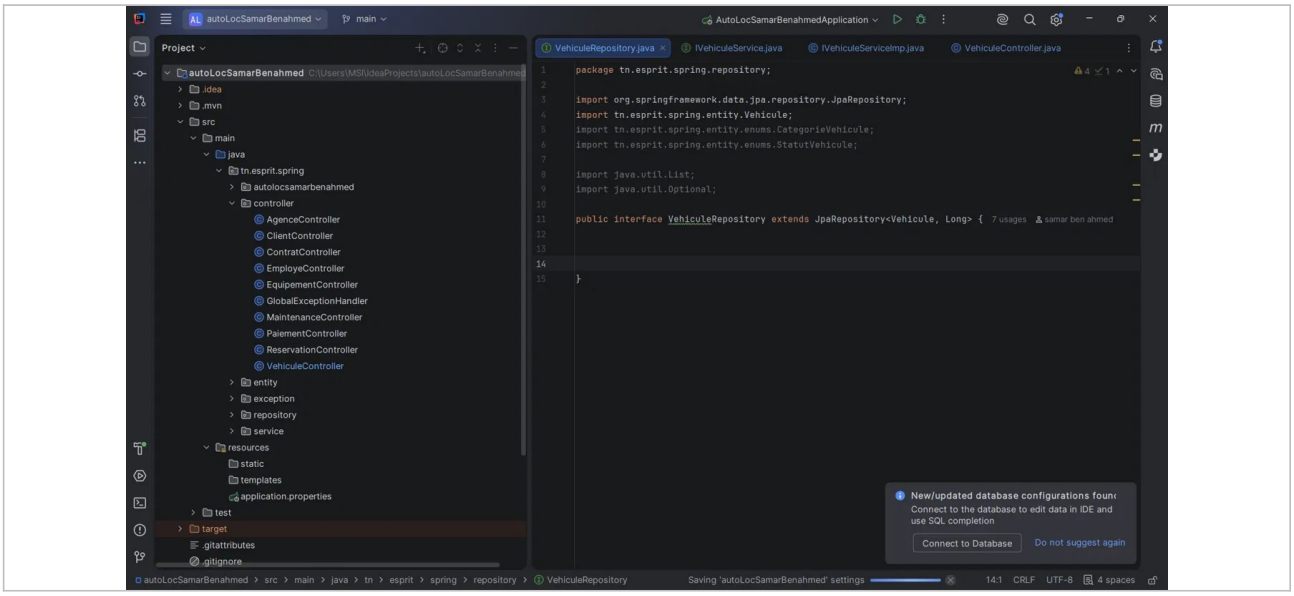


Figure A1 : VehiculeRepository.java

Explication de la capture

- **Rôle :** interface du package tn.esprit.spring.repository qui étend JpaRepository<Vehicule, Long>. Vehicule est l'entité gérée et Long le type de sa clé primaire.
- **Intérêt :** Spring Data JPA génère automatiquement l'implémentation ; les méthodes save, findById, findAll et deleteById sont disponibles sans écrire de code SQL.
- **Remarque :** le corps de l'interface est vide, aucune requête personnalisée n'étant nécessaire à ce stade. Les imports CategorieVehicule, StatutVehicule, List et Optional sont présents mais encore inutilisés (affichés en grisé).

B. IVehiculeService

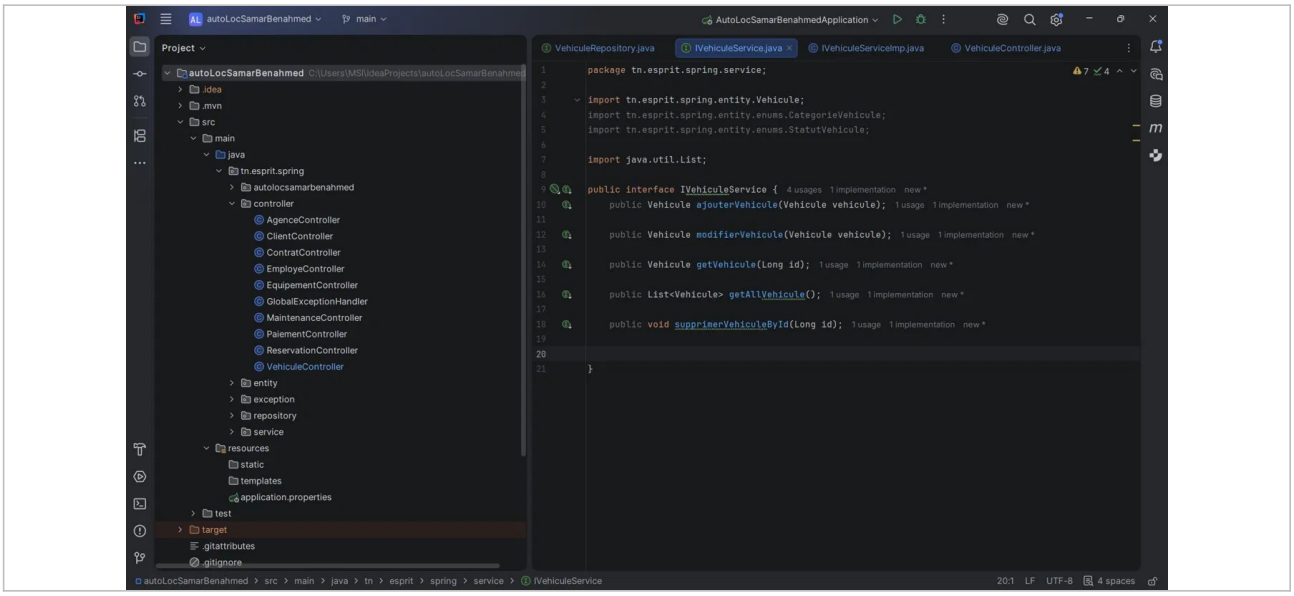


Figure A2 : IVehiculeService.java

Explication de la capture

- **Rôle :** interface du package service qui définit le contrat métier du module.
- **Méthodes déclarées :** ajouterVehicule(Vehicule), modifierVehicule(Vehicule), getVehicule(Long id), getAllVehicule() et supprimerVehiculeById(Long id).
- **Intérêt :** le contrôleur dépend de cette interface et non de l'implémentation, ce qui réduit le couplage entre les couches et facilite les tests. L'IDE indique une seule implémentation pour chaque méthode.

C. IVehiculeServiceImp

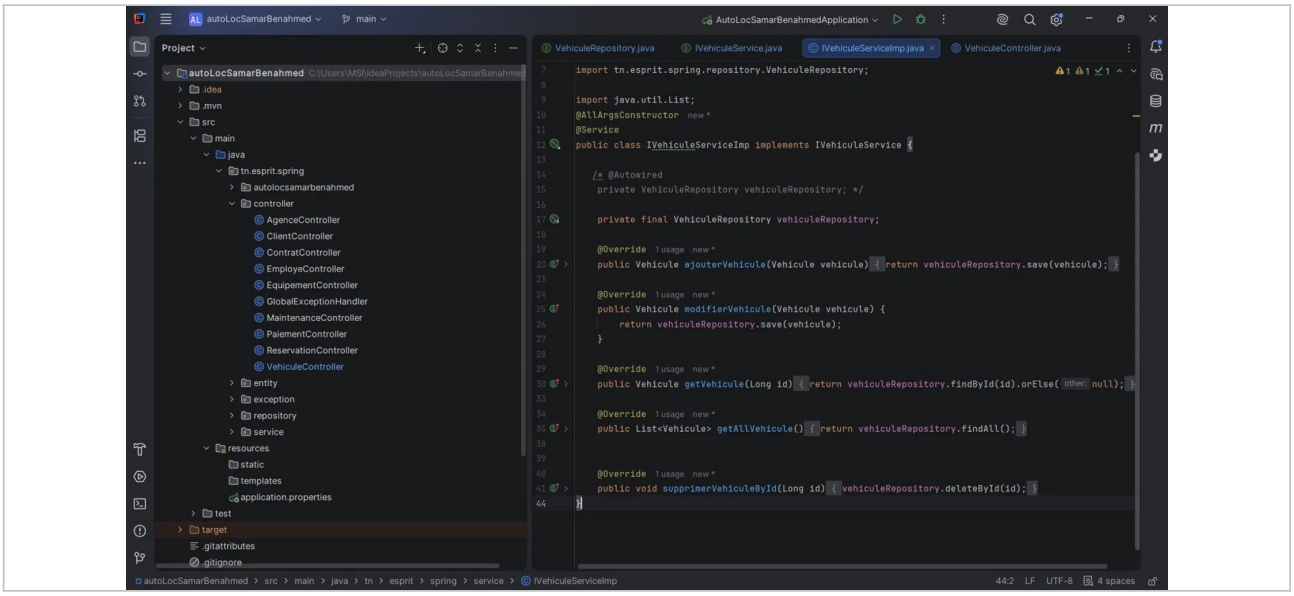


Figure A3 : IVehiculeServiceImp.java

Explication de la capture

- **Annotations :** @Service déclare la classe comme composant Spring ; @AllArgsConstructor (Lombok) génère le constructeur qui permet l'injection de VehiculeRepository, déclaré private final. L'ancienne injection par @Autowired est laissée en commentaire.
- **ajouterVehicule et modifierVehicule :** appellent toutes deux vehiculeRepository.save(vehicule). Le comportement dépend de l'identifiant : sans id, JPA insère ; avec un id existant, JPA met à jour.
- **getVehicule :** findById(id).orElse(null) retourne le véhicule ou null s'il n'existe pas.
- **getAllVehicule et supprimerVehiculeById :** utilisent respectivement findAll() et deleteById(id).

D. VehiculeController

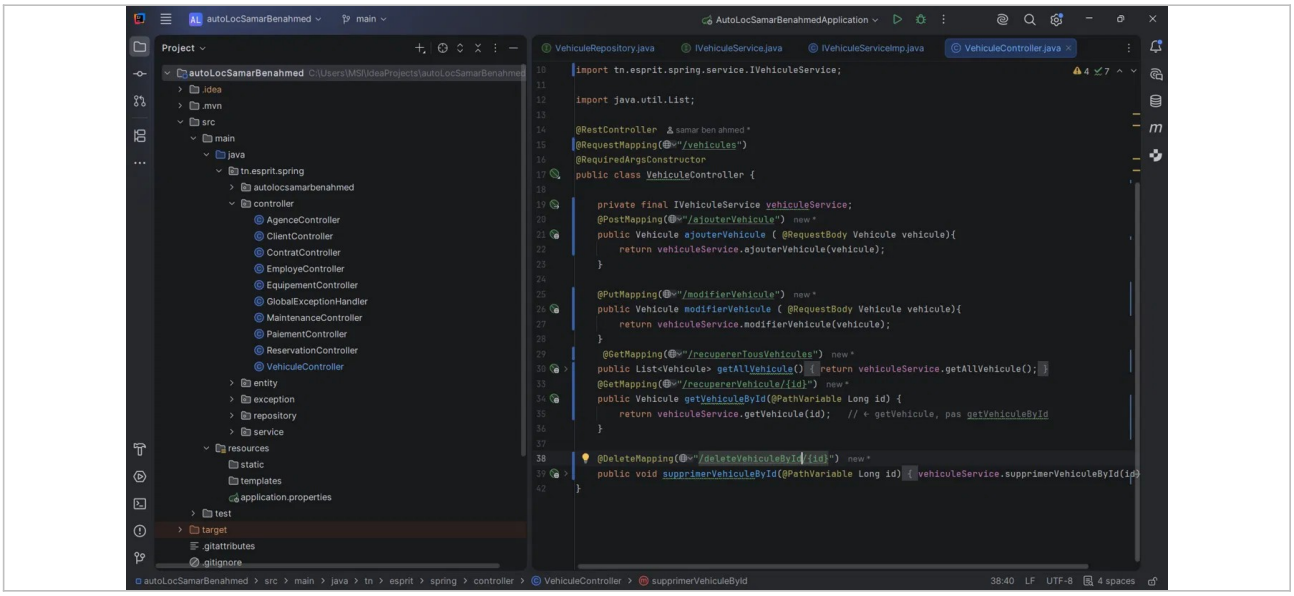


Figure A4 : VehiculeController.java

Explication de la capture

- **Annotations de classe :** @RestController (les retours sont convertis en JSON), @RequestMapping("/vehicules") (préfixe commun à tous les endpoints) et @RequiredArgsConstructor (injection par constructeur de IVehiculeService).
- **Endpoints :** cinq mappings, détaillés dans le tableau ci-dessous.

Méthode	Chemin	Méthode du service appelée
POST	/ajouterVehicule (@RequestBody)	ajouterVehicule(vehicule)
PUT	/modifierVehicule (@RequestBody)	modifierVehicule(vehicule)
GET	/recupererTousVehicules	getAllVehicule()
GET	/recupererVehicule/{id} (@PathVariable)	getVehicule(id)
DELETE	/deleteVehiculeById/{id} (@PathVariable)	supprimerVehiculeById(id)

Ces cinq mappings correspondent exactement aux URL testées avec Postman dans la section 3. Le contrôleur ne contient aucune logique métier : il délègue chaque appel à la couche service.